Project Structure

app/

- layout.tsx → Root Layout
- global.css → Tailwind css
- page.tsx → Landing page, linking both patient and staff

patient/

- page.tsx → Patient Container, session lifecycle, validation

staff/

- page.tsx → Staff Container, Fetch, Sort Order, etc...

components/

- FormField.tsx → Labelled Input plus error messages
- StatusBadge.tsx → Active, Idle, Submitted, etc...
- PatientCard.tsx → Name, Badge, Field Rows

lib/

- supabase.ts → Shared Supabase Client

supabase/

- schema.sql → Patient Table, Tirggers and Realtime Publication

Design

The form is a single column at every breakpoint, constrained to max-w-lg and centred. Intake forms are filled top to bottom, often on a phone at a reception desk or before coming to the hospital, so a second column would just add confusion to the user.

Tailwind is mobile-first, so the unprefixed class is the phone layout and each prefix overrides upward.

Green carries every action and the Active state, grey is neutral chrome.

Red is shown when there are errors or needs to fill items in.

Every control has a visible focus ring for keyboard navigation. Invalid fields get a red border.

Component Architecture

Two page containers own all state and side effects; three presentational components own rendering.

The patient container, app/patient/page.tsx, holds the session id, form values, validation errors and save status, and runs two effects: one that creates or resumes a session on load, and one that saves 500 milliseconds after the user stops typing.

The staff container, app/staff/page.tsx, holds the patient map, manages the realtime subscription lifecycle and sort order, and runs the two-second timer that lets a status decay from Active to Idle.

Below them, FormField renders one field as an input or a select and reports changes upward through onChange. PatientCard lays out a single patient and calls getStatus. StatusBadge maps a status string to its pill styling. All three are pure functions of their props, which makes them testable in isolation.

The form is driven by a FIELDS array whose name values match the database columns.

Real Time Sync Flow

1. The patient types. React updates local state and repaints that input.

2. A 500ms timer starts. Each keystroke re-runs the autosave effect, whose cleanup cancels the pending timer.

3. On expiry the browser sends an update to Supabase.

4. Postgres writes the row; a trigger stamps updated_at.

5. The change is emitted to the supabase_realtime publication.

6. Realtime pushes the row over the WebSocket the staff page opened on mount.

7. The staff page merges it into its state map by id, and React re-renders that one card.

The two pages never address each other. Status is derived, not stored. getStatus() returns Submitted if submitted_at is set, Active if updated_at is under 8 seconds old, otherwise Idle. Nothing fires an event when a patient stops typing, so the staff page re-renders every 2 seconds to let Active decay to Idle.